QUEUES
BREADTH-FIRST TRAVERSAL
COMPLETE BINARY TREES

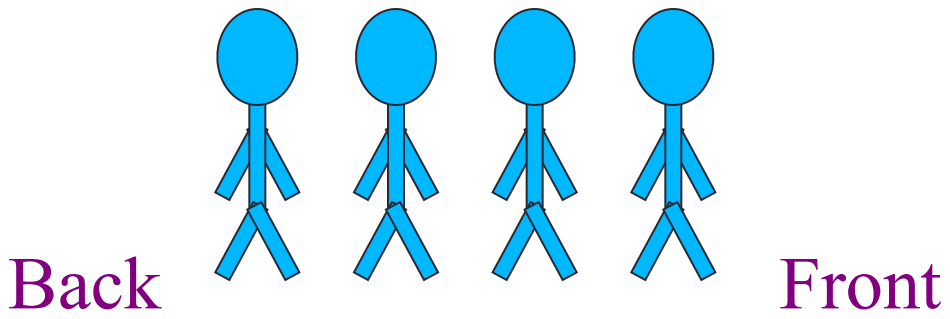
Problem Solving with Computers-II

Announcements

- Midterm next week, May 8 (Thursday)!
  - Closed book, closed notes
  - Practice problems available in Canvas
  - All topics covered so far including this week's lectures
  - Data structures covered: Linked lists, BST, stacks and queues
  - Labs 1 - 4 and pa01
  - Leetcode problem sets 1- 3

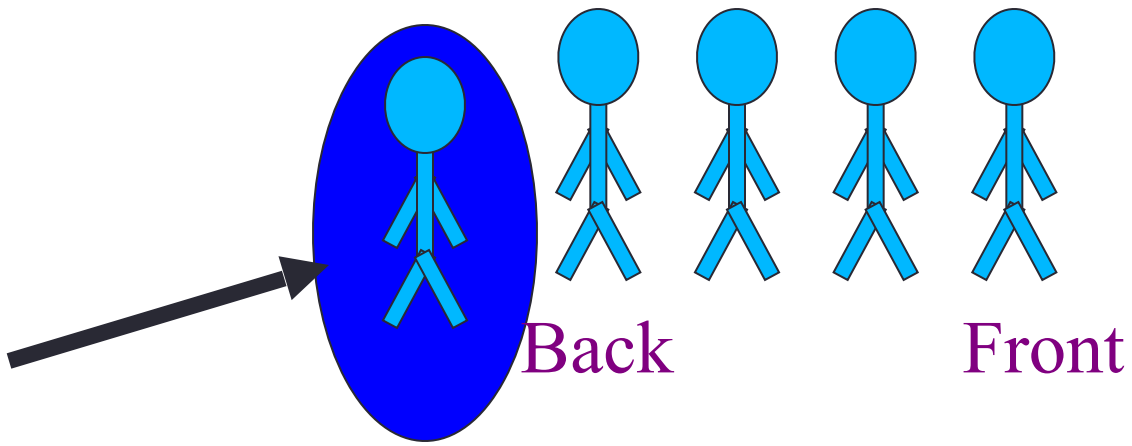
Queue

- A queue is like a queue of people waiting to be serviced
- The queue has a **front** and a **back**.



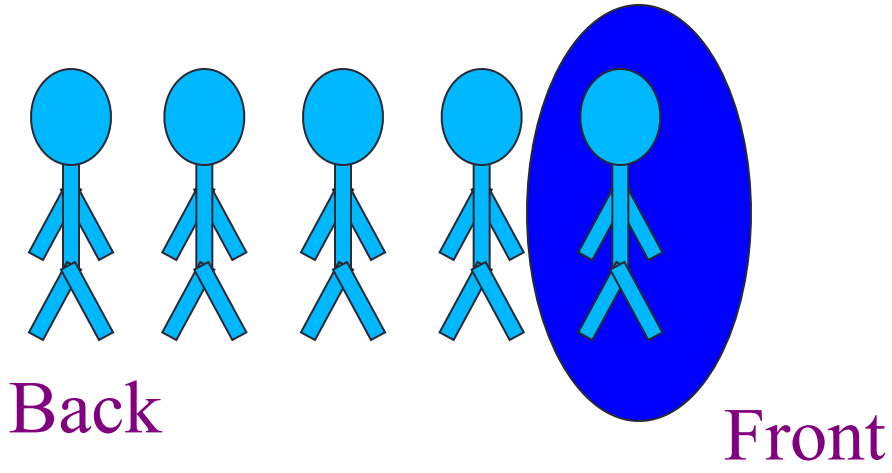
Queue Operations: push, pop, front, back

New people must enter the queue at the back. The C++ queue class calls this a **push** operation.



Queue Operations: push, pop, front, back

- When an item is taken from the queue, it always comes from the front. The C++ queue calls this a **pop**



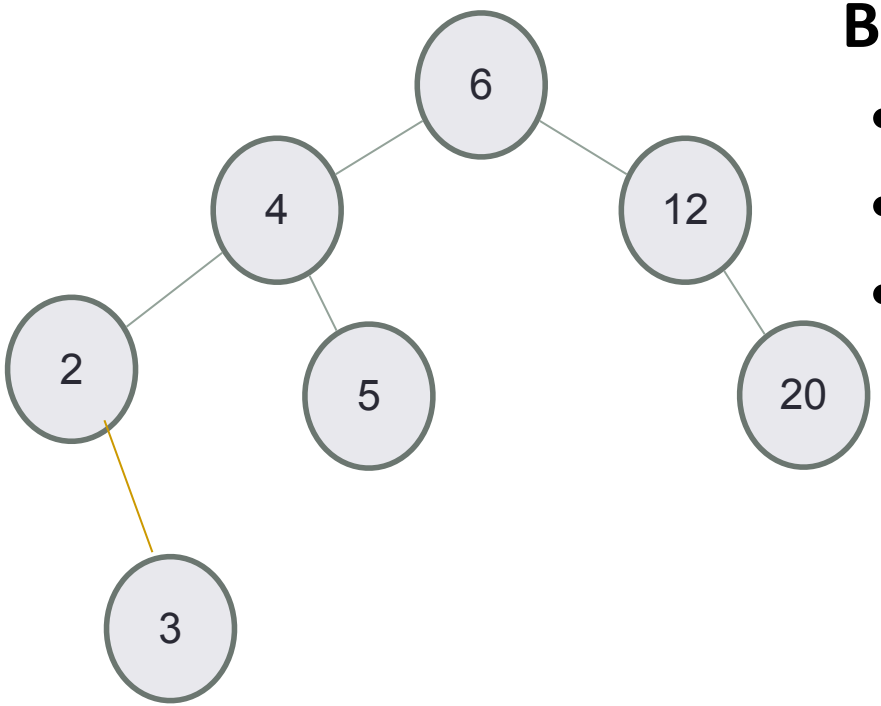
Queue class

- The C++ standard template library has a queue template class.
- The template parameter is the type of the items that can be put in the queue.

```
template <class Item>
class queue<Item>
{
public:
   queue(  );
   void push(const Item& entry);
   void pop(  );
   bool empty(  ) const;
   Item front(  ) const;
   Item back(  ) const;

};
```

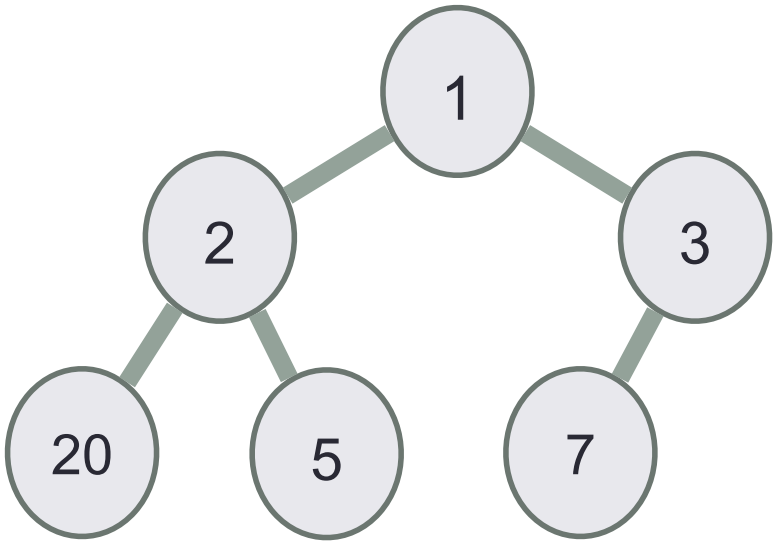
Breadth-first traversal



BFS Algo:

- Create an empty **queue**.
- Insert the **root** into the **queue**.
- While queue is not empty,
  - Insert all the children of the node into the queue.
  - Print the key in the front of the queue
  - Pop the front node from the queue

Breadth-first traversal

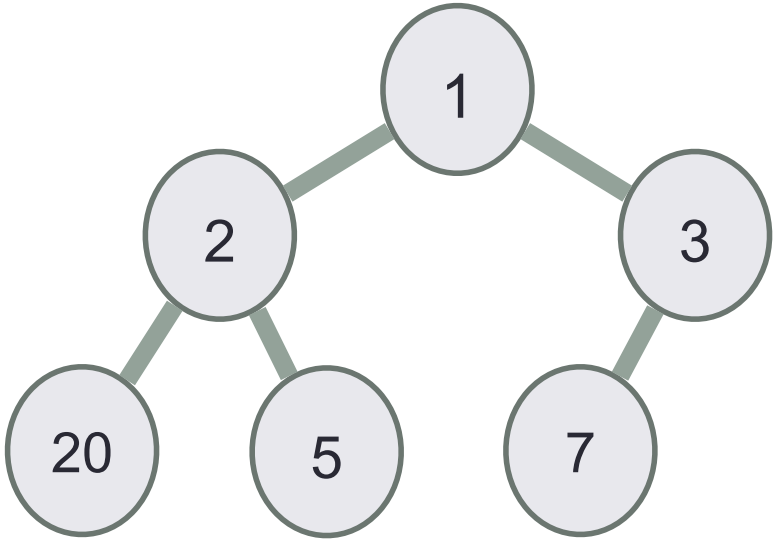


BFS Algo (store output in a vector: result):

- Create an empty **queue**.
- Insert the **root** into the **queue**.
- While queue is not empty,
  - Insert all the children of the node into the queue.
  - **Append the key in the front of the queue to result**
  - Pop the front node from the queue

Apply BFS to the tree to get the vector representation

Types of BSTs

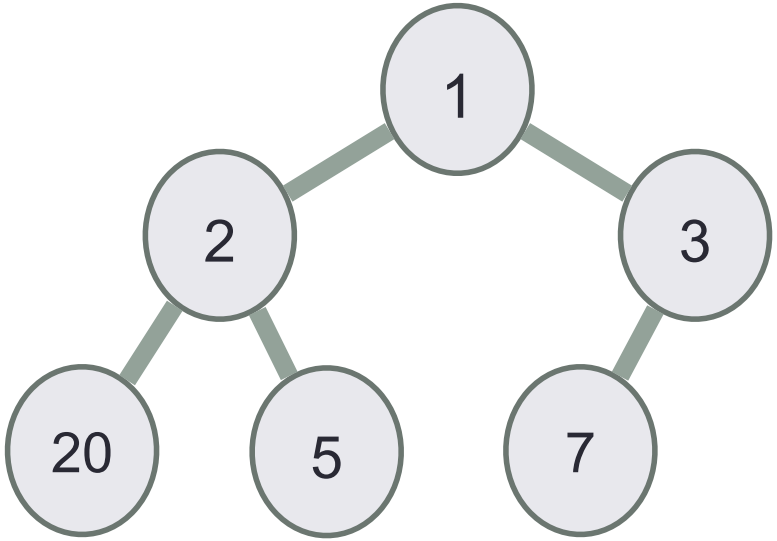


Complete Binary Tree: Every level, except possibly the last, is completely filled, and all nodes on the last level are as far left as possible

Full Binary Tree: A complete binary tree whose last level is completely filled

Complete/full binary trees are **balanced trees**!

Complete binary tree can be represented as a vector!



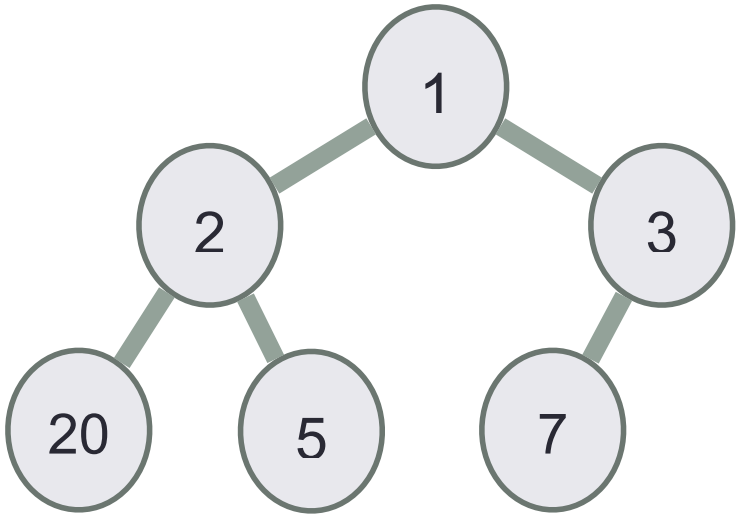
For a key at index i

index of its parent is

index of left child is

index of right child is

Complete binary tree can be represented as a vector!



index
key
parent
left child
right child

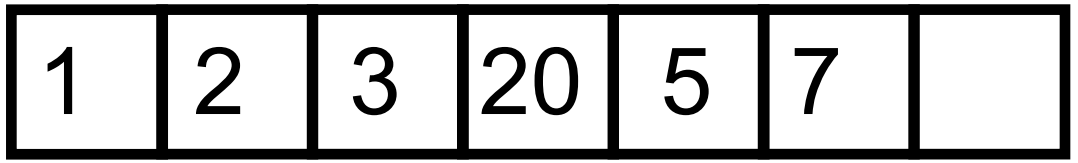
For a key at index i

index of its parent is

index of left child is

index of right child is

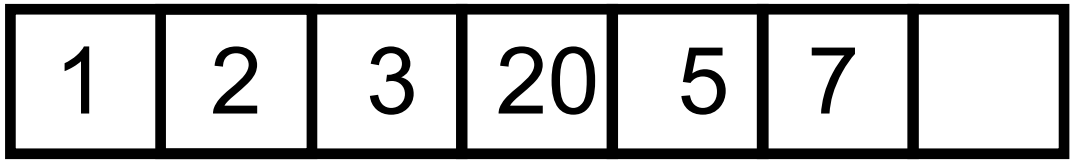
Traverse the tree only using the vector!



Based on the vector representation alone, the parent of 7 is:

A. 5
B. 3
C. 2
D. None of the above

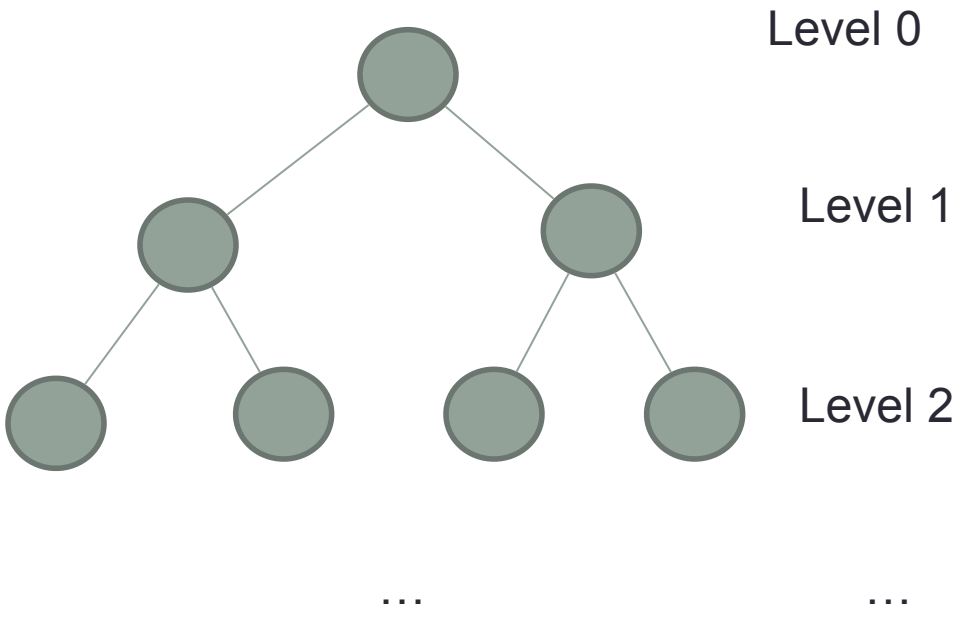
Traverse the tree only using the vector!



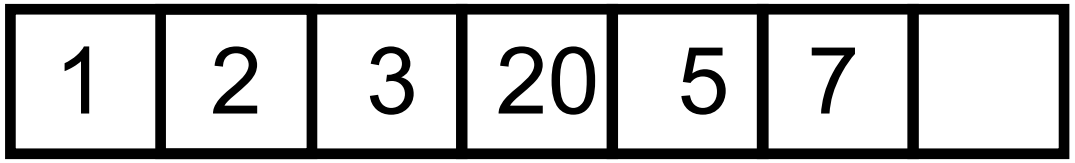
Based on the vector representation alone, find the left and right children of node with key 2:

A. (3, 20)
B. (20, 5)
C. (5, 7)
D. None of the above

Show that a complete binary tree is balanced



Add a new key (4) to the tree?



What is the complexity of adding new keys to the tree, ensuring that the resulting tree is a complete tree:

A. O(1)
B. O(log n)
C. O(n)
D. None of the above